

Part.1

프로세스 간 통신 구현

Inter-Process Communication

멀티쓰레드 · 멀티프로세스 환경에서의 데이터 송수신 기능 구현 및 검증

ToDo#1 쓰레드 간

ToDo#2 프로세스 간

ToDo#3 TCP/IP

IpcProc · C / Visual Studio 2022 · Windows x64

목차

발표는 네 단계로 진행한다 — 아래 각 항목은 해당 슬라이드의 제목과 같다

- 1

과제 개요

과제 요구사항과 구현 매핑 / 솔루션 구조

슬라이드 3 ~ 4
- 2

구현

ToDo#1 쓰레드 간 송/수신 / ToDo#1 핵심 코드 / ToDo#2 프로세스 간 송/수신 / ToDo#3 TCP/IP / 리눅스 코드를 윈도우로 옮길 때

슬라이드 5 ~ 9
- 3

동작 확인

동작 확인용 UI / 동작 확인 · 프로세스 간 송/수신 / UI 정리 · New Process 버튼 통합 / 검증 결과

슬라이드 10 ~ 13
- 4

마무리

정리 — 배운 것 세 가지와 후속 조치 결과

슬라이드 14

발표 진행

총 14매 · 약 14분 + 질의응답 (시연 시 +3분) · 과제 개요 → 구현 → 동작 확인 → 마무리

과제 요구사항과 구현 매핑

세 가지 ToDo 를 모두 같은 데이터 시나리오로 검증한다

ToDo#1

쓰레드 간 송/수신

전역변수 + 뮤텝스 / 세마포어

- 전역 링버퍼 8슬롯
- CRITICAL_SECTION 1개
- 카운팅 세마포어 2개
- 송신·수신 쓰레드 각 1개

IpThread.c

ToDo#2

프로세스 간 송/수신

메시지 큐 (Message Queue)

- 네임드 공유메모리 링버퍼
- 네임드 뮤텝스 1개
- 네임드 세마포어 2개
- 큐 이름으로 서로 참여

IpMsgQ.c

ToDo#3

다른 프로그램과 송/수신

TCP / IP

- SocketUtility.h 윈도우 포팅
- Tx = 클라이언트 (connect)
- Rx = 서버 (bind·listen·accept)
- 함수·상수명 원본 유지

IpSocket.c

공통 데이터 시나리오

송신측 1 로 초기화된 int 를 0.1초 간격으로 송신하고 송신 후 1씩 증가 (100까지) 수신측 받은 데이터를 출력

솔루션 구조

IPC 기능은 전부 C로 작성한 DLL 에 있고, MFC 는 버튼과 로그 표시만 담당한다

IpcCore

C DLL — IPC 기능 구현

IpcExternalICD.h	외부 ICD · 공통 정의 · 로그
IpcCore.c	로그 파이프라인 · 초기화 / 해제
IpcThread.c	쓰레드 간 송수신
IpcMsgQ.c	프로세스 간 송수신 (메시지 큐)
IpcSocket.c	TCP/IP (SocketUtility 포팅)



IpcUI

MFC 대화상자 — 동작 확인용

- 버튼이 코어 함수를 그대로 호출한다
- IPC 로직은 UI 에 한 줄도 두지 않았다
- 코어가 f_IpcLog 로 남긴 로그를 콜백으로 받는다
- PostMessage 로 UI 쓰레드에 넘겨 리스트박스에 출력

```
// 워커 쓰레드 -> UI 쓰레드
PostMessage(hWnd, WM_IPC_LOG, ch, pszWide);
```

빌드 결과 : build\x64\Debug\ → IpcUI.exe + IpcCore.dll

ToDo#1 쓰레드 간 송/수신

IpcThread.c

전역 링버퍼를 뮅텍스로 보호하고, 빈 슬롯 / 찬 슬롯 개수를 세마포어 2개로 센다



세마포어를 함께 쓰는 이유

```
g_hSemEmpty 초기값 8 빈 슬롯 수
g_hSemFull 초기값 0 찬 슬롯 수
```

```
// 뮅텍스만 쓰면 '빌 때까지 대기'가
// 바쁜 대기가 되어 CPU 를 태운다
```

송신 / 수신 순서

송신 sem_wait(empty) → lock → write → unlock → sem_post(full)

수신 sem_wait(full) → lock → read → unlock → sem_post(empty)

Linux 대응 : CRITICAL_SECTION = pthread_mutex_t CreateSemaphore = sem_init _beginthreadex = pthread_create

ToDo#1 핵심 코드

IpcThread.c

송신과 수신은 세마포어 두 개를 정확히 반대로 사용한다

```
f_IpcThreadQueueSend()
```

```
// ① 빈 슬롯 대기 - 가득 차면 여기서 블록
if (WaitForSingleObject(g_hSemEmpty,
    uiTimeout_ms) != WAIT_OBJECT_0)
    return IPC_FAIL;
```

```
// ② 뮉텍스로 링버퍼 상호배제
EnterCriticalSection(&g_stRingLock);
g_staRing[g_iRingTail] = *stpMsg;
g_iRingTail = (g_iRingTail + 1)
    % IPC_THREAD_RING_SLOTS;
g_iRingCount++;
LeaveCriticalSection(&g_stRingLock);
```

```
// ③ 찬 슬롯 +1 - 수신 쓰레드를 깨운다
ReleaseSemaphore(g_hSemFull, 1, NULL);
return IPC_PASS;
```

```
f_IpcThreadQueueRecv()
```

```
// ① 찬 슬롯 대기 - 비면 여기서 블록
if (WaitForSingleObject(g_hSemFull,
    uiTimeout_ms) != WAIT_OBJECT_0)
    return IPC_FAIL;
```

```
// ② 같은 뮉텍스로 상호배제
EnterCriticalSection(&g_stRingLock);
*stpMsg = g_staRing[g_iRingHead];
g_iRingHead = (g_iRingHead + 1)
    % IPC_THREAD_RING_SLOTS;
g_iRingCount--;
LeaveCriticalSection(&g_stRingLock);
```

```
// ③ 빈 슬롯 +1 - 송신 쓰레드를 깨운다
ReleaseSemaphore(g_hSemEmpty, 1, NULL);
return IPC_PASS;
```

세마포어 대기를 뮉텍스 밖에 두는 것이 핵심 — 잠근 채로 기다리면 상대가 들어올 수 없어 데드락이 된다

ToDo#2 프로세스 간 송/수신

IpcMsgQ.c

Windows 에는 System V 메시지 큐가 없어 같은 동작을 직접 만들었다

Linux

msgget(key, IPC_CREAT)

msgsnd()

msgrcv()

msgctl(IPC_RMID)

mtype

Windows 구현

CreateFileMapping 네임드 공유메모리 + 링버퍼 헤더 초기화

세마포어(빈슬롯) 대기 → 뮉텍스 → write → 세마포어(찬슬롯) 증가

세마포어(찬슬롯) 대기 → 뮉텍스 → read → 세마포어(빈슬롯) 증가

모든 핸들 CloseHandle

st_IpcMsg.iMsgType

커널 오브젝트 이름

Local\IpcProc.<큐이름>.map	공유메모리
Local\IpcProc.<큐이름>.mtx	뮉텍스
Local\IpcProc.<큐이름>.sem.e	빈 슬롯
Local\IpcProc.<큐이름>.sem.f	찬 슬롯

기동 순서에 무관하게 만든 방법

- f_IpcMsgQCreate() 는 없으면 만들고 있으면 참여한다
- 링버퍼 헤더 초기화는 뮉텍스 안에서 매직값으로 한 번만
- 세마포어 초기 카운트가 최초 생성 시에만 반영되는 Windows 동작을 그대로 이용

Global₩ 이 아니라 Local₩ 을 쓰므로 관리자 권한이 필요 없다

ToDo#3 TCP / IP

lpcSocket.c

첨부받은 Linux SocketUtility 를 Winsock2 로 포팅했다 — 함수명 · 인자 · 상수는 원본 그대로

송신 = 클라이언트

socket → connect → send

- 블로킹 connect 는 실패 확정까지 20초 이상 걸린다
- 논블로킹 + select 로 1초마다 재시도하게 바꿨다

수신 = 서버

socket → bind → listen → accept → recv

- accept 를 200ms 단위 select 루프로 잘게 나눴다
- 덕분에 [Stop] 을 누르면 대기 중에도 바로 빠져나온다

데모 송신 - 프레임 헤더 없이 int 4바이트

```
for (iData = IPC_DEMO_FIRST_VALUE;
    iData <= IPC_DEMO_LAST_VALUE; iData++) {
    iWire = g_iDemoUseNBO
        ? htonl(iData) : iData;
    f_SocketSendTCP_IPv4(&g_stDemoSocket,
        &iWire, sizeof(iWire));
    // 0.1초 대기 (정지 요청도 함께 확인)
    s_IsDemoStopRequested(100);
}
```

바이트 오더

- x64 Windows 와 x86-64 Linux 는 둘 다 little-endian 이라 기본값으로 값이 그대로 맞는다
- 상대가 htonl 을 쓰면 UI 의 htonl 체크박스를 켜서 맞춘다
- 송/수신은 요청한 크기를 다 처리할 때까지 반복한다 (>0 처리 바이트, 0 타임아웃, -1 상대 종료)

리눅스 코드를 윈도우로 옮길 때

컴파일은 통과하는데 런타임에 이상 동작하는 항목이 특히 위험하다

항목	Linux	Windows
소켓 핸들	int	SOCKET (UINT_PTR) — x64 에서 64bit
송/수신 타임아웃	setsockopt(SO_RCVTIMEO, timeval)	DWORD 밀리초 — timeval 을 넘기면 엉뚱한 값
소켓 닫기	close()	closesocket()
초기화	없음	WSAStartup / WSACleanup 필요
에러 확인	errno	WSAGetLastError()
select() 1번 인자	maxfd + 1	무시된다
마이크로초 대기	usleep()	없음 — Sleep 또는 QPC 스핀

타임아웃 값은 형이 맞아 보여서 그냥 넘어가기 쉽다 — timeval(초·마이크로초) 을 DWORD(밀리초) 로 변환하는 코드를 내부에 두었다

동작 확인용 UI

버튼 하나가 코어 함수 하나를 호출한다 — 모든 로그는 아래 리스트박스에 모인다

The screenshot shows a window titled "IpcProc" with several sections:

- Thread:** Contains "Start" and "Stop" buttons. A red circle with the number "1" is next to the "Start" button.
- Message Queue:** Contains a "Name" input field with "IpcDemoQ", and "Recv", "Send", and "Stop" buttons. A blue circle with the number "2" is next to the "Send" button.
- TCP/IP:** Contains "IP" (127.0.0.1), "Port" (51000), an "htonl" checkbox, and "Recv", "Send", and "Stop" buttons. A green circle with the number "3" is next to the "Stop" button.
- Log List:** A scrollable list box containing 16 log entries, each with a timestamp and a message (e.g., "14:32:10 [TH] rx 38"). A blue circle with the number "4" is next to the list.
- Bottom Bar:** Contains "New Process", "Clear", and "Close" buttons. A red circle with the number "5" is next to the "New Process" button.

IpcUI.exe · Debug | x64

1 Thread

[Start] 로 송신·수신 쓰레드 한 쌍을 기동한다. 동작 중에는 [Start] 가 비활성으로 바뀐다

2 Message Queue

큐 이름을 맞춘 두 프로세스가 각각 [Recv] / [Send]. 동작 중에는 이름 입력칸이 잠긴다

3 TCP/IP

IP · Port · htonl 을 정하고 한쪽은 [Recv](서버), 다른쪽은 [Send](클라이언트)

4 로그 리스트박스

세 채널의 로그가 시각과 함께 한 곳에 쌓인다. 최대 2000줄을 유지한다

5 New Process

같은 exe 를 인자 없이 한 번 더 띄운다. 프로세스 간 확인에 쓴다

동작 확인 · 프로세스 간 송/수신

ToDo#2

[New Process] 로 창을 하나 더 띄우고 한쪽은 [Recv], 다른쪽은 [Send] 를 누른다

수신 프로세스

먼저 띄운 창 · [Recv]

IpcProc

Thread

StartStop

Message Queue

NameIpcDemoQRecvSendStop

TCP/IP

IP127.0.0.1Port51000☐ htonlRecvSendStop

14:35:08 [MQ] rx 15
14:35:08 [MQ] rx 16
14:35:08 [MQ] rx 17
14:35:08 [MQ] rx 18
14:35:08 [MQ] rx 19
14:35:08 [MQ] rx 20
14:35:09 [MQ] rx 21
14:35:09 [MQ] rx 22
14:35:09 [MQ] rx 23
14:35:09 [MQ] rx 24
14:35:09 [MQ] rx 25
14:35:09 [MQ] rx 26
14:35:09 [MQ] rx 27
14:35:09 [MQ] rx 28
14:35:09 [MQ] rx 29

New ProcessClearClose

송신 프로세스

[New Process] 로 띄운 창 · [Send]

IpcProc

Thread

StartStop

Message Queue

NameIpcDemoQRecvSendStop

TCP/IP

IP127.0.0.1Port51000☐ htonlRecvSendStop

14:35:08 [MQ] tx 15
14:35:08 [MQ] tx 16
14:35:08 [MQ] tx 17
14:35:08 [MQ] tx 18
14:35:08 [MQ] tx 19
14:35:08 [MQ] tx 20
14:35:09 [MQ] tx 21
14:35:09 [MQ] tx 22
14:35:09 [MQ] tx 23
14:35:09 [MQ] tx 24
14:35:09 [MQ] tx 25
14:35:09 [MQ] tx 26
14:35:09 [MQ] tx 27
14:35:09 [MQ] tx 28
14:35:09 [MQ] tx 29

New ProcessClearClose

같은 큐 이름 **IpcDemoQ** 만 맞으면 어느 쪽을 먼저 띄워도 된다 — `f_IpcMsgQCreate()` 가 없으면 만들고 있으면 참여하기 때문

UI 정리 · New Process 버튼 통합

역할별로 흩어져 있던 버튼을 공용 버튼 하나로 줄여 과제의 핵심 기능만 남겼다

변경 전

- Message Queue 와 TCP/IP 에 [New Process] 버튼이 각각 하나씩
- 버튼이 상대 역할을 스스로 정해 명령행으로 넘겼다
- 새 창이 그 역할을 자동으로 시작했다

```
CIpcUIDlg::RunPeer()
strCmd.Format(_T("\'%s\' /peer:%s /name:%s"
    " /ip:%s /port:%u%s"), szExePath,
    lpszRole, m_strQueueName, m_strIpAddr,
    m_nPortNum, m_bNetworkByteOrder
    ? _T(" /nbo") : _T(""));

// + ParseIpcArguments() 명령행 파서
// + OnInitDialog 자동 시작 분기
```

변경 후

- 공용 [New Process] 버튼 하나만 하단에 둔다
- 인자 없이 같은 exe 를 한 번 더 띄우는 것이 전부
- 역할은 사용자가 새 창에서 직접 누른다

```
CIpcUIDlg::OnBnClickedNewProcess()

TCHAR szExePath[MAX_PATH] = { 0 };
GetModuleFileName(nullptr, szExePath,
    MAX_PATH);

CreateProcess(szExePath, nullptr, nullptr,
    nullptr, FALSE, 0, nullptr, nullptr,
    &si, &pi);
```

결과 버튼 2개 · 명령행 파서 · 자동 시작 분기 제거 → UI 코드 84줄 감소, 과제가 요구한 IPC 기능은 그대로

검증 결과

세 가지 모두 1 부터 100 까지 0.1초 간격으로 송신하고 수신측이 같은 값을 출력한다

쓰레드 간

ToDo#1

```
[TH] start
[TH] tx 1
[TH] rx 1
[TH] tx 2
[TH] rx 2
[TH] tx 3
[TH] rx 3
...
[TH] tx 100
[TH] rx 100
[TH] tx done (100)
[TH] rx done (100)
```

프로세스 간

ToDo#2

```
[MQ] queue 'IpcDemoQ' open
[MQ] start (rx)
[MQ] rx 1
[MQ] rx 2
[MQ] rx 3
...
[MQ] rx 99
[MQ] rx 100
[MQ] stop
[MQ] rx done (100)
```

TCP / IP

ToDo#3

```
[TCP] listen 0.0.0.0:51000
[TCP] accept 127.0.0.1:60312
[TCP] rx 1
[TCP] rx 2
[TCP] rx 3
...
[TCP] rx 99
[TCP] rx 100
[TCP] peer closed
[TCP] rx done (100)
```

확인한 것

- ① 100건이 순서대로 빠짐없이 도착한다 ② 수신측이 대기 중에도 [Stop] 에 바로 반응한다 ③ 두 프로세스의 기동 순서를 바꿔도 동작한다

정리

1

무텍스와 세마포어는 짝이다

상호배제는 무텍스, 개수를 세고 기다리는 것은 세마포어. 하나로 다른 하나를 흉내내면 바쁜 대기나 데드락이 된다

2

메시지 큐는 없으면 만들 수 있다

공유메모리 링버퍼 + 네임드 무텍스 + 세마포어 2개면 msgsnd / msgrcv 와 같은 의미를 그대로 만들 수 있다

3

포팅에서 위험한 것은 타입이 맞는 차이

SO_RCVTIMEO 처럼 컴파일은 통과하는데 값의 의미가 달라지는 항목을 먼저 찾아야 한다